

Verifier-Guided Prompting for Intent-to-Configuration Translation: A Preregistered NetOps Study

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

Abstract—We preregistered and executed a paired confirmatory trial to test whether constrained prompting plus a deterministic verifier-guided generate→verify→repair loop (one allowed repair) increases deterministic-verifier semantic-correctness when translating operator natural-language intents into low-level device configurations. Design: N=150 synthetic intents sampled from a deterministic generator, shared_initial_candidate generation semantics, model = gpt-5-mini, deterministic validator = deterministic_netops_validator_v5, one initial generation per intent shared across baseline and guarded conditions, guarded pipeline permitted ≤1 repair call. Results (artifact-grounded): baseline = 149/150 (99.333%); guarded = 150/150 (100.0%); paired absolute difference = 0.006667 (≈0.67 percentage points); exact McNemar two-sided p = 1.0; 95% bootstrap percentile CI = [0.00, 0.02] (10,000 resamples, seed = 1729). Provider accounting: completed episodes = 300; model_calls_used = 151; repair-stage invocations = 1. Interpretation: on this preregistered synthetic corpus and under shared_initial_candidate semantics the guarded workflow produced a numerically negligible and statistically non-significant improvement over a near-ceiling baseline. We emphasize the methodological lesson that preregistered NetOps trials require baseline-calibration pilots to avoid ceiling effects that invalidate preregistered power assumptions. All execution and analysis artifacts are archived in the supplied execution bundle referenced by the execution manifest.

I. INTRODUCTION

Natural-language intent interfaces aim to reduce operator burden by letting humans express desired network changes as free text that is automatically translated into executable device configurations. Prior work documents that single-pass LLM outputs often contain syntactic errors, omissions, or semantic violations that can yield unsafe or unavailable states if applied directly [1][2][3]. A commonly proposed defense is a generate→verify→repair architecture in which an LLM produces a candidate, a deterministic verifier checks intent-level invariants or formalized constraints, and an automated repair (or human gate) corrects violations before execution [4][5]. Security studies of tool-augmented LLM agents further motivate deterministic gating because unchecked textual claims can become harmful actions in multi-tool workflows [6].

Research question. After an autonomous literature review and preregistration we posed a confined confirmatory question: does constrained prompting (structured templates with explicit semantic slots) combined with deterministic verifier-guided iterative feedback materially increase verifier-level semantic correctness of NL→configuration translation versus

an unconstrained baseline under a paired design? The trial (study_cand_2026_b2_v1) was preregistered and frozen before any model outputs were observed.

Contributions. (1) A fully preregistered, artifactized paired confirmatory trial measuring deterministic-verifier pass/fail on a programmatically generated synthetic corpus (N=150 paired intents). (2) Artifact-backed outcomes showing that, under shared_initial_candidate semantics and a one-repair budget, the guarded pipeline raised verifier pass rate from 149/150 to 150/150 (+0.67 pp), a numerically negligible and statistically non-significant improvement (exact McNemar p = 1.0; 95% bootstrap CI = [0.00, 0.02]). (3) A methodological recommendation that preregistered NetOps trials include baseline-calibration pilots and explicitly specify generation semantics (shared_initial_candidate vs independent sampling) because semantics materially change the estimand and interpretation.

II. RELATED WORK

Related literature falls into three strands; each strand provides partial motivation for the guarded generate→verify→repair architecture we evaluate and clarifies where prior claims and limitations remain.

Intent-to-configuration translation. A growing body of work explores converting operator natural-language intents into formal specifications or concrete device configurations. Empirical evaluations and surveys report that single-pass translation often attains only modest semantic-correctness on curated benchmarks, motivating post-generation checks and constrained prompting to improve fidelity [1][2]. Representative studies highlight varied task formulations (intent grammars, template-based translation, and intent-slot extraction) and report that success rates depend strongly on task granularity, the expressiveness of the target formalism, and prompt design choices [1]. Systematic reviews and targeted evaluations underscore that many NL→formal pipelines still produce semantic errors that formal verifiers can detect but that the end-to-end reliability required for unattended NetOps remains an open problem [2]. These findings justify using deterministic verifiers as the primary confirmatory endpoint and motivate task-bank designs that capture common operational idioms (reachability, ACL-like constraints, VLAN/MTU sequences, and uplink switchover patterns) rather than only isolated syntactic transforms.

Generate→verify→repair architectures and verifier technology. Several recent method papers and systems integrate deterministic or policy-guided verifiers with generation to improve correctness for infrastructure-as-code and configuration-repair tasks [4][7]. Approaches range from using verifier feedback for fine-tuning to runtime repair loops that iteratively correct a candidate until invariants hold. Reported improvements are heterogeneous: gains are larger when baseline model competence is modest, when larger repair budgets or multiple independent samples are allowed, or when verifier feedback provides actionable, fine-grained corrections rather than coarse pass/fail signals [4][7]. Critically, the effectiveness of verifier-guided repair depends on verifier coverage and the fidelity of the mapping layer that translates natural-language intents to verifier inputs; prior work emphasizes rigorous unit-testing of mapping code and staged integration with formal tools [5].

A complementary thread documents mature formal verification tools (e.g., NetKAT variants, KATch, Hoyan, and other symbolic verifiers) that can act as deterministic oracles for reachability, isolation, and policy checks in controlled topologies [5][9]. These tools vary in supported abstractions, latency, and scale: some are optimized for design-time proofs while others (e.g., recent NetKAT provers and scalable overlay checkers) demonstrate faster runtime checks for smaller topologies. Prior studies therefore treat verifier selection and verifier-coverage testing as design decisions that directly affect measured semantic-correctness and external validity.

Security, prompt-injection, and claim-to-action risks. Security- and attack-oriented analyses show that LLM-driven multi-tool workflows can convert false textual claims into concrete tool calls and harmful actions unless guarded by auditable gates or hardened planning primitives [6][8]. Empirical prompt-injection studies demonstrate practical exploitation strategies and optimization-based attacks against LLM-as-judge setups; corresponding defenses (prompt hardening, verifier-aware gating, and call-auditing) reduce risk but impose trade-offs with automation throughput [6][8]. These results motivate conservative guarded architectures that retain verifier traces and normalized configurations for operator review, a design choice we make explicit when framing operational interpretation beyond raw pass/fail rates.

How this study connects and differs. The present trial operationalizes a narrowly specified guarded architecture informed by the above strands: we evaluate verifier-level pass/fail on a deterministic synthetic corpus (so verifier selection and mapping tests are central), we limit repair budget to a single repair call to match a constrained operational budget, and we adopt `shared_initial_candidate` semantics to isolate the marginal effect of repair applied to an identical first-pass candidate. These design choices intentionally expose how sampling semantics, verifier coverage, and repair budgets interact with baseline competence—issues emphasized across the cited literature—and explain why previously reported larger guarded-vs-baseline gains can co-exist with a near-ceiling outcome in tightly controlled, verifier-capable synthetic settings [4][5][7].

III. METHODOLOGY

Overview and preregistration. We preregistered `study_cand_2026_b2_v1` and froze an analysis plan prior to any model calls. The design is paired-binary: each sampled intent yields one shared initial candidate used to evaluate both the baseline and guarded conditions (`shared_initial_candidate` semantics). The primary estimand was the `paired_success_rate_difference_guarded_minus_baseline` and the confirmatory test was an exact two-sided McNemar on the paired contingency table. The preregistration specified $N=150$ confirmatory intents, inclusion/exclusion rules, a one-repair budget for guarded episodes, a deterministic retry policy for provider timeouts (one retry), and an analysis plan that included a 10,000-resample paired bootstrap (`bootstrap_seed = 1729`) to produce percentile CIs. These preregistered elements are archived; the registered preregistration SHA-256 is recorded in the execution manifest.

Task generation, intent families, and prechecks. Confirmatory tasks were produced deterministically by `deterministic_netops_task_generator_v5`. The generator emits intent payloads with: (i) an explicit natural-language intent string, (ii) an `initial_state` and `expected_final_state`, (iii) an ordered `required_sequence` of field-level operations (e.g., `shutdown→mtu→restore`), (iv) policy constraints (`allowed_touched_fields`, `protected_interfaces`, `final_group_constraints`), and (v) a declared difficulty label. The confirmatory corpus was balanced across four intent families: safe sequence reachability/isolation, ACL-like access changes, VLAN/MTU sequence manipulations, and uplink switchover patterns. Topologies were restricted to verifier-capable small networks (4–32 nodes) to ensure deterministic validator coverage. Prior to model calls we ran deterministic unit-tests on the mapping/transformation code and a rule-based ambiguity detector; items flagged as ambiguous or failing parser/unit-tests were excluded per the preregistered exclusion rules. In this execution none of the confirmatory items were excluded.

Execution adapter, sampling semantics, and explicit baseline interpretation. The `hosted_netops_gvr_v1` adapter enforced the `shared_initial_candidate` semantics required by the preregistration: for each intent the adapter performed exactly one initial model generation (the shared candidate) and used that identical candidate when scoring both the baseline and guarded conditions. The guarded pipeline could issue at most one additional repair call only if the deterministic validator failed the shared initial candidate. Because of this architecture, the baseline success rate measures the validity of the single initial sampled candidate; the guarded vs baseline contrast therefore quantifies the marginal effect of a single allowed repair applied to the identical sampled candidate rather than the effect of independently re-sampling first-pass candidates per condition. We include this precise sampling-to-estimand mapping in Methods to eliminate interpretive ambiguity about whether different first-pass generations were drawn for each condition.

Model configuration and nondeterminism. Declared

model: gpt-5-mini (provider record: openai_responses). The archived execution/model_configuration.json records declared_model_version = "gpt-5-mini" and explicitly records temperature = null (unset). Importantly, the adapter and preregistration did not enforce a numeric temperature value or a fixed random seed; therefore model outputs may be nondeterministic across independent API sessions. We state this here in Methods and repeat it in the Disclosure Statement to comply with preregistration/runtime-parameter reporting requirements.

Repair budget, session isolation, and provider auditing. The guarded repair policy allowed at most one repair invocation per intent and the adapter maintained strict session isolation: each model call ran in a fresh API session (no cross-call state reuse). Provider auditing tracked hosted_model_call_event_count and cache statistics; the deterministic reconciliation artifact reports completed_hosted_model_call_event_count = 151, cache_miss_count = 151, and stage_counts = {shared_initial_generation: 150, repair: 1}. The adapter implementation enforces the preregistered retry rule (one retry after transient provider failure). All provider-call metadata and per-call runtime parameters were archived for audit.

Deterministic validator, scoring rule, and diagnostic trace structure. We used deterministic_netops_validator_v5 (validator_version = 5.0) as a deterministic oracle for the primary binary endpoint. The validator implements a full_intent_constraint_validation rule that: (a) parses the LLM-produced candidate into normalized command sequences, (b) computes parsed_command_count and compares against required_command_count derived from the task's required_sequence, (c) enforces workflow and group constraints (e.g., make-before-break, must_be_down_for_fields), and (d) returns a boolean valid flag plus structured diagnostics (violation_codes, observed_touched_field_count, normalized_configuration, final_state snapshot). Scoring for the confirmatory endpoint used the validator's boolean pass/fail; the richer diagnostics were used for failure-mode analysis and are archived as validator traces for each episode.

Inclusion/exclusion, contamination controls, and pre-execution unit-tests. The preregistered inclusion rules required that an intent pass the deterministic ambiguity detector and that mapping code successfully produce verifier inputs. We executed deterministic unit-tests for mapping code and verifier-input transformations before batch evaluation; these tests passed. Contamination checks (session-leak detection, duplicate-response entropy checks) were run; contamination_summary.csv recorded zero flags. Deterministic reconciliation confirms episode accounting consistency and reports no provider-call failures.

Statistical analysis posture (confirmatory vs exploratory). The methodology adheres to the preregistered multiplicity and missingness rules: the primary confirmatory contrast is guarded vs baseline at $\alpha=0.05$ (exact McNemar two-sided) using complete_pair_primary (exclude incomplete pairs). Secondary contrasts were preregistered (constrained vs baseline,

guarded vs constrained) with Bonferroni correction; in practice shared_initial_candidate semantics and adapter limits constrained the realized secondary contrasts. Analysis artifacts include paired_contingency_table.csv, condition_summary.csv, and a paired bootstrap used to compute the 95% CI (bootstrap_resamples = 10000, bootstrap_seed = 1729). All analysis code, seeds, and artifacts are archived for reproducibility.

Reproducibility artifacts and archival pointers. The full execution manifest, raw model-call logs, per-call runtime metadata, task-generator seeds, verifier inputs/verdicts, and analysis outputs are archived in the execution bundle referenced by the execution manifest. Deterministic reconciliation and provider accounting artifacts record model_calls_used = 151 and stage-level counts; these artifacts support replication of the paired accounting and the precise sampling semantics used in this trial.

IV. RESULTS

Execution and provider audit. The run completed as planned: completed_episode_count = 300 and complete_pair_count = 150. Provider-call accounting: model_calls_used = 151; completed_hosted_model_call_event_count = 151; failed_hosted_model_call_event_count = 0; cache_miss_count = 151. Stage-level counts: shared_initial_generation = 150 and repair = 1. No contamination flags or pre-execution unit-test failures occurred.

Primary numerical outcomes and contingency counts. The paired contingency counts (guarded $\times$ baseline) were: $n_{11} = 149$ (both-pass), $n_{10} = 1$ (guarded-only-pass), $n_{01} = 0$ (baseline-only-pass), $n_{00} = 0$ (both-fail). Per-condition success rates: baseline_success_count = 149/150 = 0.9933333333333333; guarded_success_count = 150/150 = 1.0. Estimated paired absolute difference (guarded - baseline) = 0.00666666666666671 (≈ 0.67 percentage points). Exact McNemar two-sided $p = 1.0$. Paired bootstrap percentile 95% CI = [0.00, 0.02] (10,000 resamples; bootstrap_seed = 1729). These figures are reported verbatim from the archived analysis artifacts (analysis/results.json, analysis/paired_contingency_table.csv, analysis/condition_summary.csv).

Repair diagnostics and revision iterations. Only one guarded repair invocation occurred (1/150), consistent with the near-ceiling initial success rate. That single repair converted one initially invalid candidate to a valid configuration under the deterministic verifier. Median revision iterations for guarded episodes = 0 (IQR = 0). Validator traces include validation_before and validation_after states, normalized_configuration strings, parsed_command_count, required_command_count, violation_codes, and a valid flag for each episode; representative traces for tasks 000001–000006 are archived and demonstrate varied difficulty patterns while confirming validator behavior.

Representative artifact-grounded example. Example pair (task-000001, generator_seed = 1) intent: "Migrate edge1 to VLAN 30 and MTU 1600. For safety, shut edge1

down before changing either VLAN or MTU, then restore it to admin up. Do not modify any other interface." Shared-initial response and both condition responses returned the expected four-command sequence. Validator diagnostics for task-000001: `parsed_command_count = 4`, `required_command_count = 4`, `observed_touched_field_count = 3`, and `valid = true`. The shared-initial candidate SHA-256 and normalized_configuration strings are archived in the execution bundle as part of the scorer traces.

Sensitivity and missingness. The preregistered sensitivity analysis treating missing guarded outputs as failures was not invoked because `failed_hosted_model_call_event_count = 0` and `unscorable_episodes = 0`. No exclusions or parse failures occurred and `contamination_summary.csv` is empty.

Compact evidence tables. Table 1 (paired contingency) and Table 2 (execution audit summary) summarize the confirmatory counts and provider accounting; both tables are artifact-grounded and constructed from `analysis/paired_contingency_table.csv` and `analysis/condition_summary.csv`.

Interpretation of numeric outcome. On this synthetic corpus and under `shared_initial_candidate` semantics the guarded workflow produced a numerically negligible absolute improvement (+0.67 pp) from an already near-ceiling baseline. The exact McNemar $p = 1.0$ and the bootstrap CI containing zero indicate no statistically supported confirmatory benefit under the preregistered estimand. Importantly, the observed baseline success rate ($\approx 99.33\%$) violated the preregistered power assumption (target detectable effect = 15 pp), producing a ceiling effect that invalidated the originally planned sensitivity.

V. DISCUSSION

Ceiling effect and implications for preregistration. The preregistered power calculation assumed a substantially lower baseline; `observed_baseline_success_rate = 0.9933` left almost no headroom for the guarded repair to show a large absolute improvement. This ceiling effect invalidated the preregistered detectable-effect assumptions (target = 15 percentage points) and reduced the trial’s ability to demonstrate benefit. Concretely, with $n_{11} = 149$ and only a single discordant pair ($n_{10} = 1$), the paired analysis is dominated by an almost-complete concordance that leaves little information for estimating discordant behavior. We therefore recommend that preregistered NetOps trials include a short baseline-calibration pilot (for example, 20–40 tasks sampled from the planned generator and prompt style) to estimate baseline competence and either increase planned N or raise task difficulty before committing to confirmatory sampling. The archived deterministic task generator seeds and representative tasks permit such a pilot: one can sample the same generator with a small pilot, measure `baseline_success_rate`, and then choose confirmatory N or difficulty stratification so that the expected baseline leaves statistical headroom for the target absolute improvement.

Why the synthetic corpus produced a high baseline. The deterministic generator produced tasks with declared difficulty markers (medium, hard, very_hard) and explicit

required_sequence constraints; inspection of representative traces (tasks 000001–000006) shows many tasks have fully specified required sequences and constrained action spaces (`allowed_fields`, `allowed_touched_fields`) that reduce translation ambiguity. When intents are tightly specified and the verifier coverage matches the required invariants, even a single sampled candidate from a capable LLM often satisfies the verifier. In our run this interaction produced the near-ceiling baseline, illustrating that generator design choices (ambiguity, under-specification, and difficulty-sampling) materially influence the observable marginal benefit of any repair stage.

Semantics, sampling, and estimand trade-offs. The execution semantics (`shared_initial_candidate`) intentionally reduced per-intent sampling variance by using the same initial candidate across conditions; this made the guarded-baseline contrast a pure marginal-repair estimand. Alternative semantics— independent per-condition sampling or ensembles of initial draws—estimate different operational questions: independent sampling measures expected verifier pass rate when each pipeline draws independently from the model distribution, while multi-draw or multi-repair budgets measure how re-sampling or repeated repairs trade cost for improved correctness. Independent sampling or larger repair budgets typically increase the chance of observing discordant pairs (higher $n_{10} + n_{01}$) and therefore can increase power to detect smaller absolute differences, but at the cost of additional model calls and latency. Practitioners should select sampling semantics that match deployment constraints (whether systems will resample or must act on a single first-pass candidate) and incorporate model-call cost and repair-latency budgets into experimental power planning.

Repair budgets, cost, and diminishing returns. The provider accounting from our run (`model_calls_used = 151`; `repair-stage_invocations = 1`) quantifies the operational cost of the guarded design under the one-repair budget: repairs were rarely invoked when the baseline was near-ceiling, so marginal cost per observed success-gain was effectively high. In scenarios with lower baseline competence, repair budgets exhibit diminishing returns: initial repairs convert many failures, but additional repairs yield progressively fewer conversions. Quantifying diminishing returns requires multi-arm experiments that vary allowed repair counts and measure per-repair marginal gain versus model-call cost; our artifacts supply seeds and code to run such arms reproducibly.

Operational affordances beyond pass/fail. Deterministic guarded workflows provide benefits not captured by a single binary endpoint. The validator produces `normalized_configuration` strings, `parsed_command_count`, `required_command_count`, and explicit `violation_codes` that operators can audit; these artifacts accelerate triage and human review even when verifier-pass differences are numerically small. For example, representative validator traces in the archive show concrete `violation_code` semantics (missing step, order-violation) that would guide automated or human repair policies. Thus, guarded pipelines can improve operational observability, reduce human triage time, and support auditable

change records independent of marginal verifier-pass improvements on constrained corpora.

Threats to validity and residual risks. Internal validity: adapter-enforced call budgets and the provider audit mitigate cross-condition leakage, and mapping unit-tests reduced parse/translation failures. Construct validity: deterministic validator pass/fail is a proxy for semantic correctness and depends on mapping code and verifier coverage; although mapping unit-tests passed and no parse failures occurred, any uncovered verifier-coverage gap would bias semantic-correctness measurement. External validity: experiments used a single declared model (gpt-5-mini) and synthetic tasks on small topologies (4–32 nodes); results should not be extrapolated to larger, heterogeneous networks or other models. Security and adversarial robustness: this confirmatory trial did not evaluate adversarial prompt-injection; prior work indicates claim-to-action attacks remain an independent concern and defensive evaluation requires separate adversary-driven experiments. Conclusion validity: the mismatch between preregistered detectable-effect assumptions and the observed baseline underscores the importance of pilot calibration to preserve inferential validity.

Practical experimental prescriptions. Based on artifact-grounded outcomes we recommend: (1) a baseline-calibration pilot (20–40 tasks) to estimate baseline_success_rate and discordant-pair proportions prior to final N selection; (2) explicit preregistration of generation semantics and repair budgets; (3) multi-arm designs that vary repair budgets and sampling semantics to estimate cost–benefit frontiers; (4) inclusion of provider-call accounting in power and cost planning; and (5) staged scaling of verifier coverage and topology size with deterministic unit-tests for mapping code. The archived execution manifest and representative tasks provide concrete seeds and runnable code to implement these prescriptions reproducibly.

VI. LIMITATIONS

- Single-model constraint: experiments used only gpt-5-mini; results do not generalize across models or sizes.
- Synthetic corpus and scale: tasks were programmatically generated and limited to verifier-capable toy-to-small topologies (4–32 nodes); external validity to production WANs and heterogeneous vendor CLIs is untested.
- Shared_initial_candidate semantics and execution contract limited repair budget to at most one guarded repair call per intent; different generation semantics or multiple-repair budgets may yield different outcomes.
- Ceiling effect and preregistration mismatch: baseline correctness was far higher than the pilot assumptions used for power calculations (planned detectable effect = 15 percentage points), reducing the trial’s ability to demonstrate benefit and illustrating sensitivity to baseline calibration.
- Verifier coverage and specification translation: the study measures deterministic validator pass/fail on the prereg-

istered invariants but does not claim safety guarantees beyond the deterministic validator’s modeled properties.

- Security and adversarial robustness: the experiment did not evaluate adversarial prompt-injection or adversary-driven intents; separate targeted studies are required to assess claim-to-action vulnerabilities in agentic NetOps workflows.

VII. CONCLUSION

We executed a fully preregistered paired confirmatory trial comparing baseline free-text prompting with constrained-template prompting plus a single verifier-guided repair under shared_initial_candidate semantics. On a deterministic synthetic corpus with gpt-5-mini and deterministic_netops_validator_v5, the guarded pipeline increased verifier pass rate from 149/150 to 150/150 (≈ 0.67 pp), a numerically tiny and statistically non-significant difference (exact McNemar $p = 1.0$; 95% bootstrap CI = [0.00, 0.02]). The principal scientific lesson is methodological: preregistered NetOps trials should include baseline calibration to avoid ceiling effects and preserve statistical power. Technically, our artifacts bound the marginal benefit of a one-repair guarded pipeline under shared_initial_candidate semantics; evaluating independent-sampling semantics, larger repair budgets, model diversity, and production-scale topologies remains important future work.

DISCLOSURE STATEMENT

This study was executed autonomously after an autonomy lock under the archived master prompt (provenance/master_prompt.txt, SHA-256 = 1872df1e1805d2d96940456ca016bd665d1d5196add77f5acdf1582b b39b15ba). Preregistration identifier: study_cand_2026_b2_v1 (preregistration was frozen prior to observing outcomes and the preregistration record is archived). Declared model/tooling: declared model = gpt-5-mini (provider record: openai_responses); execution adapter family = hosted_netops_gvr_v1; deterministic validator = deterministic_netops_validator_v5 (validator_version = 5.0). Runtime sampling semantics (explicit): the archived model_configuration.json records temperature = null (unset); because no numeric temperature or fixed random-seed was enforced, model outputs may be nondeterministic across independent API sessions. Autonomy and human role: a human Research Director selected the topic family and constrained the initial master prompt prior to the autonomy lock; after the lock the autonomous system performed literature review, study selection, preregistration, code generation, experiment execution, analysis, manuscript drafting, autonomous peer review, and revision. No human manually edited technical content, analyses, figures, captions, or references after the autonomy lock. Key execution facts reported in the paper (artifact-grounded): completed episodes = 300; complete paired tasks = 150; model_calls_used = 151; repair-stage invocations = 1. All runtime sampling metadata, provider-call traces, verifier inputs/verdicts, and analysis

artifacts are archived in the execution bundle referenced by the execution manifest.

REFERENCES

- [1] <https://doi.org/10.1002/nem.2313>
- [2] <https://doi.org/10.1145/3786583.3786898>
- [3] <https://arxiv.org/abs/2604.22513>
- [4] <https://doi.org/10.1145/3605764.3623985>
- [5] <https://doi.org/10.1145/3656454>
- [6] <https://doi.org/10.1002/widm.70111>